FINAL REPORT

ZK LOGIN IN CARDANO

Project #1400130

Name of the project: ZK Login in Cardano - Eryx
Project number: #1400130
Name of project manager: Agustin Franchella
Date project started: Nov 24, 2025
Date project completed: Mar 30, 2026

List of challenge KPIs and how the project addressed them

1. **Enabling end-user growth in the Cardano community:**
   zkLogin replaces private key management with familiar Web2 login (Google OAuth), removing the primary friction point for onboarding new users. The protocol preserves privacy while offering self-custody and a login experience identical to Web2 apps. This directly addresses the adoption gap between Web2 users and Cardano.
2. **Privacy-preserving authentication for Cardano users:**
   The protocol guarantees two core properties: security (unforgeability — no adversary can sign transactions on behalf of the user) and unlinkability (no third party can link on-chain address to off-chain identity). The adversarial model ensures no single actor (wallet, OIDP, salt service) has enough information to compromise the user.
3. **Integration of Zero-Knowledge Proofs with Cardano smart contracts:**
   zkLogin demonstrates a production-grade use of ZK proofs on Cardano. The Aiken validator verifies Groth16 proofs on-chain to authenticate users via OpenID credentials, proving that the aiken-zk tooling (from the prior Catalyst grant #1300084) can support real-world applications. The project implements Circom circuits for JWT signature verification (RSA), Poseidon hashing (nonce and zkLoginId derivation), and JWT payload parsing — all verified on-chain within Cardano's 16 KB transaction limit.
4. **Cross-compatibility with ecosystem components:**
   The project integrates across multiple ecosystem tools: Circom for circuit definition, snarkjs for proof generation, aiken-zk for on-chain Groth16 verification, Mesh SDK and Cardano Serialization Library for transaction building, Blockfrost for chain queries, and Google's OpenID Connect for authentication.
5. **Cross-chain protocol adaptation to Cardano:**
   The project adapts the academically published zkLogin protocol (originally implemented natively in Sui's core protocol) to Cardano's UTxO model using Aiken validator scripts. This demonstrates that Cardano's smart contract layer is flexible enough to support protocols originally designed for account-based chains.
6. **Advancing self-custody without compromising user experience:**
   Traditional self-custody wallets require users to safeguard private keys, which is error-prone and leads to asset loss. zkLogin provides a third option: the user controls their funds through a combination of OIDC authentication and a secret salt, neither of which alone is sufficient to sign transactions. This preserves non-custodial guarantees while removing the burden of key management, since the user can safely rely on a third party without the usual risks.
7. **Complete Circom circuit suite for the zkLogin protocol:**
   Seven circuits were implemented: RSA signature verification, JWT payload parsing, zkLoginId derivation, nonce derivation, and supporting arithmetic circuits (BigInt, modular exponentiation), all composed in a main circuit.

List of project KPIs and how the project addressed them

1. **Working end-to-end zkLogin flow on Cardano testnet:**
   A complete end-to-end zkLogin implementation was built and demonstrated on the Cardano Preview testnet. Users can log in with Google, derive a zkLogin address, generate a ZK proof, receive funds via a custom faucet, and send ADA to any Cardano address — all verified on CardanoScan.
2. **End-to-end proof workflow validated:**
   The full pipeline works: off-chain credential generation (ephemeral keys, nonce, zkLoginId) → Google OAuth authentication → Groth16 proof generation (JWT signature verification + identity derivation + nonce derivation, all in-circuit) → on-chain Aiken validator verification → successful transaction execution. This was tested through unit tests (backend/src/*.test.ts), deployment tests (backend/deployment/*.test.ts), and circuit tests (backend/circuit_tests/).
3. **Documentation and community engagement:**
   A detailed 17-page technical report (report/report.pdf) was produced covering the full protocol specification, adversarial model, design decisions (why Groth16, signature algorithms, hash functions), wallet integration responsibilities, advantages/disadvantages, differences from the Sui implementation, and future contribution roadmap. The README provides step-by-step instructions for running and testing the project.
4. **Frontend application with guided user experience:**
   A React + Vite application delivers an 8-step guided flow covering the full protocol: ephemeral credential creation, randomness generation, nonce derivation, Google authentication, salt creation/recovery, address derivation, proof generation, faucet funding and funds transfer.
5. **Developer accessibility and tooling usability:**
   The project provides a complete reference implementation that any wallet or dApp developer can study and integrate. It includes a React frontend with an 8-step guided user flow, a TypeScript backend with clear API endpoints (derive address, generate proof, fund address, transfer funds), and comprehensive documentation. This significantly lowers the barrier for developers wanting to add zkLogin to their Cardano applications.

Key achievements (in particular around collaboration and engagement)

- Successfully built the first implementation of zkLogin on Cardano, adapting the academically published protocol (originally from Sui) to work within Cardano's architecture using Aiken validator scripts instead of native protocol changes.
- Demonstrated that the aiken-zk tooling (from the predecessor grant #1300084) is capable of supporting a complex, real-world cryptographic application.
- Bridged three communities: Web2 authentication (OpenID/OAuth), ZK cryptography (Circom/Groth16), and Cardano smart contracts (Aiken/Plutus V3).

Key learnings

- Groth16 is currently the only viable proving system for on-chain verification on Cardano due to the 16 KB transaction size limit (FRI-based proofs start at ~17 KB) and current implementations.
- Cardano's script-based address model requires a workaround, compared to Sui's native protocol-level integration. In this case, the main workaround is embedding the user zkLoginId as a validator parameter.

Next steps for the product or service developed

- Support additional OpenID Providers (Apple, Meta, Microsoft, GitHub) by handling different JWT claim formats, signature algorithms (ES256, PS256), and JWK rotation mechanisms.
- Implement multi-device and recovery mechanisms: salt recovery flows, device migration strategies, and optional secure storage in device hardware (TPM, Secure Enclave).
- Handle OIDC public key rotation automatically with caching of JWK sets and circuit-level support for multiple accepted provider keys.
- Enable staking rewards for zkLogin addresses by exploring hybrid addresses (zkLogin payment script + conventional stake key) or stake scripts enforcing the same identity constraints.
- Remove the dependency on a backend server by moving proof generation to the user's device (browser-based WASM proving), improving both privacy (preserves unlinkability) and decentralization.
- Integrate Tx3 for transaction deployment to simplify transaction building and reduce boilerplate.
- Reduce proof generation time (currently ~2 minutes) by optimizing circuit size, exploring GPU-accelerated proving, using proof aggregation techniques or using a cloud server.
- Eliminate the custom faucet requirement by adapting Mesh SDK or transaction building to handle UTxO datum requirements transparently.
- Generalize the zkLoginId derivation to be wallet-independent (using only `sub` + `iss` instead of `sub` + `aud` + `iss`) so users can access the same address from different dApps.
- Publish the project as a reusable library/SDK that wallet developers can integrate with minimal configuration, similar to how aiken-zk provides a CLI workflow.

Final thoughts/comments

In this grant, we successfully implemented the **zkLogin protocol**, which we believe is a crucial step toward Cardano’s mass adoption. By leveraging **Zero-Knowledge Proofs**, we enable end users to access Web3 intuitively and privately, removing technical barriers and bridging the gap between ecosystems. We also want to highlight how our **aiken-zk** tooling accelerated the

development of these privacy-preserving features, marking a significant achievement for our team.

Links to other relevant project sources or documents

- **GitHub repository:** https://github.com/eryxcoop/zklogin-aiken
- **How to use:** https://github.com/eryxcoop/zklogin-aiken/blob/main/README.md
- **Documentation:** https://github.com/eryxcoop/zklogin-aiken/blob/main/report/report.pdf

Close-out video

- **YouTube:** https://www.youtube.com/watch?v=9ne0bPUXMg8